- In Programming, data type is an attribute associated with a piece of data that tells a computer system how to interpret its value.
- Understanding data types ensures that data is collected in the preferred format and that the value of each property is as expected.

What are Data Types in Programming?

- *An attribute that identifies a piece of data and instructs a computer system on how to interpret its value is called a data type.*
- The term "data type" in software programming describes the kind of value a variable possesses and the kinds of mathematical, relational, or logical operations that can be performed on it without leading to an error.
- Numerous programming languages, for instance, utilize the data types string, integer, and floating point to represent text, whole numbers, and values with decimal points, respectively.
- An interpreter or compiler can determine how a programmer plans to use a given set of data by looking up its data type.

The data comes in different forms.
Examples include:

- your name – a string of characters
- your age – usually an integer
- the amount of money in your pocket- usually decimal type
- today's date – written in date time format

Common Data Types in Programming:

1. Primitive Data Types:

- *Primitives are predefined data types that are independent of all other kinds and include basic values of particular attributes, like text or numeric values. They are the most fundamental type and are used as the foundation for more complex data types. Most computer languages probably employ some variation of these simple data types.*

2. Composite Data Types:

Composite data types are made up of various primitive kinds that are typically supplied by the user. They are also referred to as user-defined or

non-primitive data types. Composite types fall into four main categories: semi-structured (stores data as a set of relationships); multimedia (stores data as images, music, or videos); homogeneous (needs all values to be of the same data type); and tabular (stores data in tabular form).

3. User Defined Data Types:

A user-defined data type (UDT) is a data type that derived from an existing data type. You can use other built-in types already available and create your own customized data types.

Common Primitive Data Types in Programming:

Some common primitive datatypes are as follow:

Data Type	Definition	Examples
Integer (int)	represent numeric data type for numbers without fractions	300, 0 , -300
Floating Point (float)	represent numeric data type for numbers with fractions	34.67, 56.99, -78.09
Character (char)	represent single letter, digit, punctuation mark, symbol, or blank space	a , 1, !
Boolean (bool)	True or false values	true- 1, false- 0
Date	Date in the YYYY-MM-DD format (ISO 8601 syntax)	2024-01-01
Time	Time in the hh:mm:ss format for the time of day, time since an event, or time interval between events	12:34:20

Data Type	Definition	Examples
Datetime	Date and time together in the YYYY-MM-DD hh:mm:ss format	2024 -01-01 12:34:20

Common Composite Data Types:

Some common composite data types are as follow:

Data Type	Definition	Example
String (string)	Sequence of characters, digits, or symbols—always treated as text	hello , ram , i am a girl
array	List with a number of elements in a specific order—typically of the same type	arr[4]= [0 , 1 , 2 , 3]
pointers	Blocks of memory that are dynamically allocated are managed and stored	*ptr=9

Common User-Defined Data Types:

Some common user defined data types are as follow:

Data Type	Definition	Example
Enumerated Type (enum)	Small set of predefined unique values (elements or enumerators) that can be text-based or numerical	Sunday -0, Monday -1

Data Type	Definition	Example
Structure	allows to combining of data items of different kinds	struct s{ ...}
Union	contains a group of data objects that can have varied data types	union u {...}

Static vs. Dynamic Typing in Programming:

Characteristic	Static Typing	Dynamic Typing
Definition of Data Types	Requires explicit definition of data types	Data types are determined at runtime
Type Declaration	Programmer explicitly declares variable types	Type declaration is not required
Error Detection	Early error detection during compile time	Errors may surface at runtime
Code Readability	Explicit types can enhance code readability	Code may be more concise but less explicit
Flexibility	Less flexible as types are fixed at compile time	More flexible, allows variable types to change

Characteristic	Static Typing	Dynamic Typing
Compilation Process	Requires a separate compilation step	No separate compilation step needed
Example Languages	C, Java	Python, JavaScript, Ruby

Type Casting in Programming:

- Converting a single data type value—such as an integer int, float, or double—into another data type is known as typecasting. You have the option of doing this conversion manually or automatically. The conversion is done in two ways, automatically by the compiler and manually by a programmer.
- Type casting is sometimes known as type conversion. For example, a programmer can type cast a long variable value into an int if they wish to store it in the program as a simple integer. Thus, type casting is a technique that allows users to utilize the cast operator to change values from one data type to another.
- Type casting is used when imagine you have an age value, let's say 30, stored in a program. You want to display a message on a website or application that says "Your age is: 30 years." To display it as part of the message (a string), you would need to convert the age (an integer) to a string.
- Simple explanation of type casting can be done by this example:
- Imagine you have two types of containers: one for numbers and one for words. Now, let's say you have a number written on a piece of paper, like "42," and you want to put it in the container meant for words. Type casting is like taking that number, converting it into words, and then putting it in the container for words. Similarly, in programming, you might have a number (like 42) stored as one type, and you want to use it as if it were another type (like a word or text). Type casting helps you make that conversion.

Types of Type Casting:

The process of type casting can be performed in two major types in a C program. These are:

- **Implicit** – done internally by compiler.
- **Explicit** – done by programmer manually.

Syntax for Type Casting:

<datatype> variableName = (<datatype>) value;

Example of Type Casting:

1. Converting int into double

- C++
- Python3

```
#include <iostream>
using namespace std;

int main() {
int intValue = 54;
double doubleValue;
doubleValue = intValue;
cout << "int value: " << intValue << endl;
cout << "double value: " << doubleValue << endl;
return 0;
}
```

Output

```
int value: 54
double value: 54
```

2. Automatic Conversion of double to int:

- C++
- Python3

```cpp
#include <iostream>
using namespace std;

int main() {
double doubleValue = 54.7;
int intValue;
intValue = doubleValue;
cout << "double value: " << doubleValue << endl;
cout << "int value: " << intValue << endl;
return 0;
}
```

Output

```
double value: 54.7
int value: 54
```

Variables and Data Types in Programming:

The name of the memory area where data can be stored is called a variable. In a program, variables are used to hold data; they have three properties: name, value, and type. A variable's value may fluctuate while a program is running.

Data type characterizes a variable's attribute; actions also rely on the data type of the variables, as does the data that is stored in variables. The sorts of data that a variable can store are specified by its data types. Numerous built-in data types, including int, float, double, char, and bool, are supported by C programming. Every form of data has a range of values that it can store and a memory usage limit.

Example: Imagine a box labeled "age." You can put a number like 25 in it initially, and later, you might change it to 30. A box labeled "number" is designed for holding numbers (like 42). Another box labeled "name" is designed for holding words or text (like "John"). So, in simple terms, a variable is like a labeled box where you can put things, and the data type is like a tag on the box that tells you what kind of things it can hold. Together, they help

the computer understand and manage the information you're working with in a program.

- **Pointers** are one of the core components of the C programming language.
- A pointer can be used to store the **memory address** of other variables, functions, or even other pointers. The use of pointers allows low-level memory access, dynamic memory allocation, and many other functionality in C.

What is a Pointer in C?

- *A pointer is defined as a derived data type that can store the address of other C variables or a memory location.*
- *We can access and manipulate the data stored in that memory location using pointers.*

As the pointers in C store the memory addresses, their size is independent of the type of data they are pointing to.
This size of pointers in C only depends on the system architecture.

Syntax of C Pointers

The syntax of pointers is similar to the variable declaration in C, but we use the **(*) dereferencing operator** in the pointer declaration.

```
datatype * ptr;
```

where

- **ptr** is the name of the pointer.
- **datatype** is the type of data it is pointing to.

The above syntax is used to define a pointer to a variable. We can also define pointers to functions, structures, etc.

How to Use Pointers?

The use of pointers in C can be divided into three steps:

1. **Pointer Declaration**
2. **Pointer Initialization**
3. **Pointer Dereferencing**

1. Pointer Declaration

In pointer declaration, we only declare the pointer but do not initialize it. To declare a pointer, we use the **(*) dereference operator** before its name.

Example

```
int *ptr;
```

The pointer declared here will point to some random memory address as it is not initialized. Such pointers are called wild pointers.

2. Pointer Initialization

Pointer initialization is the process where we assign some initial value to the pointer variable. We generally use the **(&) addressof operator** to get the memory address of a variable and then store it in the pointer variable.

Example

```
int var = 10;
int * ptr;
ptr = &var;
```

We can also declare and initialize the pointer in a single step. This method is called **pointer definition** as the pointer is declared and initialized at the same time.

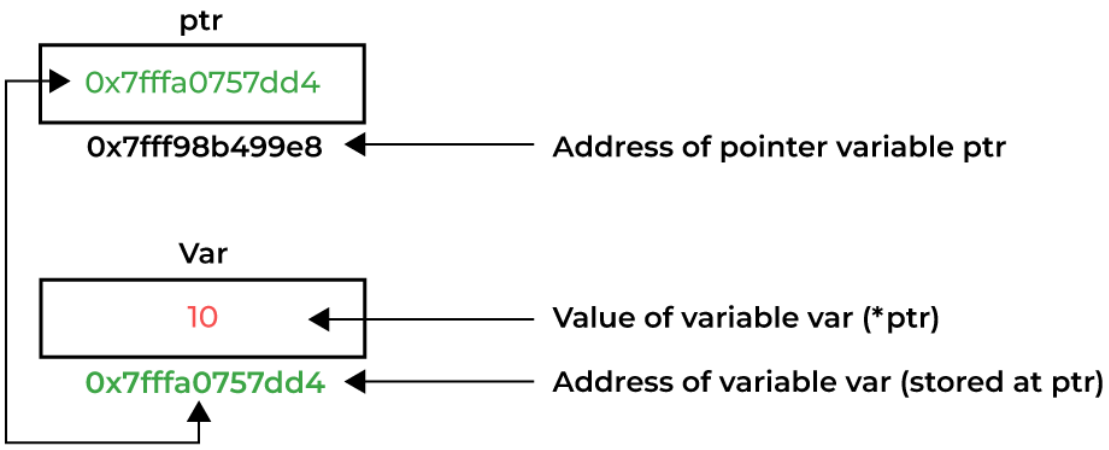
Example

```
int *ptr = &var;
```

Note: *It is recommended that the pointers should always be initialized to some value before starting using it. Otherwise, it may lead to number of errors.*

3. Pointer Dereferencing

Dereferencing a pointer is the process of accessing the value stored in the memory address specified in the pointer. We use the same **(*) dereferencing operator** that we used in the pointer declaration.



Dereferencing a Pointer in C

Coercion

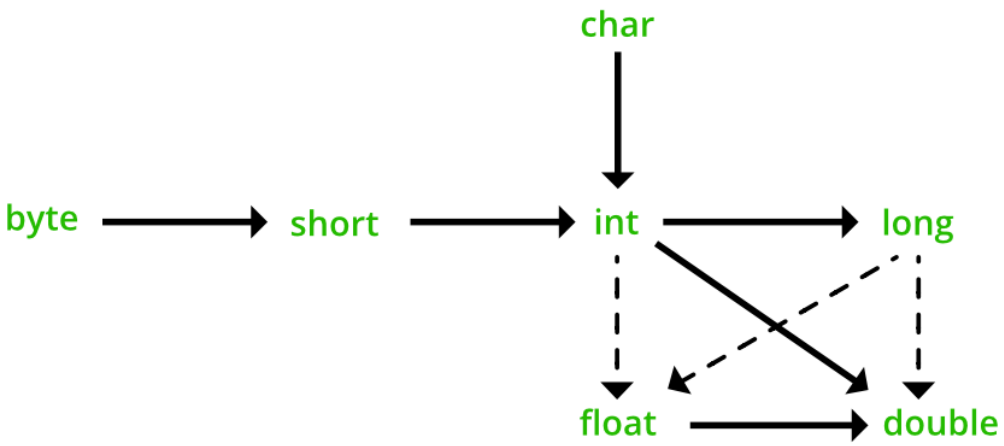
- Many programming languages support the conversion of a value into another of a different data type.
- This kind of type conversions can be implicitly or explicitly made. Implicit conversion, which is also called coercion, is automatically done.
- Explicit conversion, which is also called casting, is performed by code instructions. This code treats a variable of one data type as if it belongs to a different data type.

- The languages that support implicit conversion define the rules that will be automatically applied when primitive compatible values are involved.
- The C code below illustrates implicit and explicit coercion. In line 2 the int constant 3 is automatically converted to double before assignment (implicit coercion). An explicit coercion is performed by involving the destination type with parenthesis, which is done in line 3.

```
double x, y;
x = 3;             // implicitly coercion (coercion)
y = (double) 5;    // explicitly coercion (casting)
```

Coercion in Java

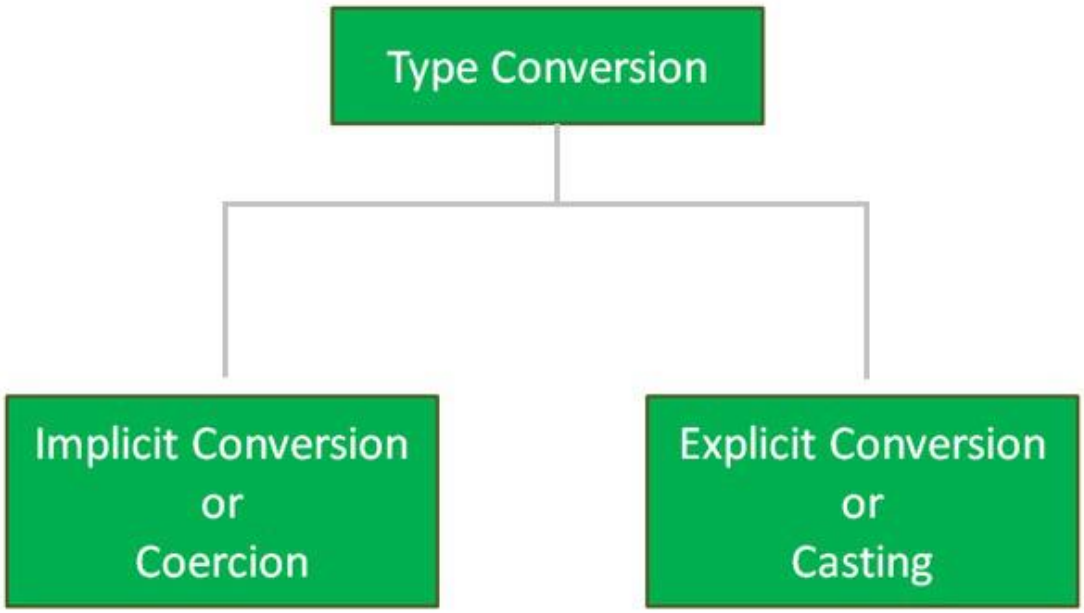
Conversion of one data type to another is nowadays easy as many languages including **Java** supports the conversion as a convenience to the programmer. Here conversion can be done in two ways i.e. **implicit** or **explicit** conversion.



- **implicit conversion:** This type of conversion is automatically done by the programming languages. In the case of Java, it is done by the JVM (Java Virtual Machine).
- **explicit conversion:** This type of conversion has to be done by the programmer as per their requirements.

Coercion in Java

here we will be discussing **coercion** *(also known as type conversion).* So, Coercion is the process of converting one type of data type to another. In a simple way, **implicit conversion** holds the title of **coercion in java.**



Example 1

- Java

```
// Java Program to Illustrate Coercion Via
// Integer  Data Type to Double  Data Type

// Importing required classes
import java.io.*;

// Class
class GFG {

    // Main driver method
    public static void main(String[] args)
    {
```

```
        // Declaring and initializing
        // integer and double variables
        int numerator = 15;
        double result = numerator / 5;

        // Print statement
        System.out.println("Result is in double format : "
                           + result);
    }
}
```

Output

```
Result is in double format : 3.0
```

In the above example, the constant (numerator) 15 is an integer but its context requires a double value.

Output explanation: Machine operation is performed on data at the execution time of the program are:

- Fetches the value of the **numerator** variable.
- Use of the literal value i.e. **5**.
- Converts the value of the literal value to a floating point.
- Performs floating-point division (CPU task).
- Stores the result into memory allocated to the **result** variable.

Example 2

- Java

```java
// Java Program to Illustrate Coercion Via
// Character data type to Integer data type

// Importing required classes
import java.io.*;

// Class
class GFG {

    // Main driver method
    public static void main(String[] args)
    {

        // Declaring character and integer variables
        char alphabet = 'A';
        int ascii_value = alphabet;

        // Print statement
        System.out.println("ASCII Value of A is: "
                           + ascii_value);
    }
}
```

Output

```
ASCII Value of A is: 65
```

In the above example, the **ascii_value** is an **integer** but its context requires a **character** value therefore with the help of **coercion/implicit** conversion it is possible with ease.

example3

int a=10,b=3;

float c=a/b;

solve a/b first, then the value of c=3

since c is float result is 3.0

example 4

int a=10,b=3;

float c=(float)a/b;

to evaluate this expression (float)a/b ,float is having higher precedence, so it will be 10.0/3.0=3.33

Tip: *Difference between Type Casting and Type Conversion?*
Casting is the process by which we treat an object type as another type while coercing is the process of converting one object to another. There are two ways to cast the primitive data type, widening, and narrowing. It is also known as implicit type casting because it is done automatically.